

Практична робота №1. Інтеграція зовнішнього REST API

Мета: Інтегрувати зовнішній REST API у веб-застосунок: запити з параметрами, обробка JSON, кодів відповіді, таймаутів і помилок, відокремлення інтеграції в окремий модуль. Підхід далі застосовується для роботи з AI API. Мова — Python (requests, FastAPI).

Зміст

1. Теоретичні відомості
2. Завдання до виконання
3. Критерії приймання
4. Контрольні запитання
5. Рекомендована література

1. Теоретичні відомості

Перш ніж викликати AI-сервіси, корисно впевнено інтегрувати звичайний зовнішній REST API: там ті самі задачі — параметри, JSON, коди відповіді, таймаути й помилки. У цій роботі ви інтегруєте публічний погодні API у веб-застосунок. Підхід, який тут вибудуєте, знадобиться далі для роботи з AI API.

Матеріал вам знайомий — нижче лише опора, щоб виконувати роботу свідомо.

Запит і параметри. Звернення до REST API — це HTTP-запит на URL із параметрами (найчастіше в `query-рядку`). Параметри варто передавати структуровано (окремим набором «ключ — значення»), а не склеювати рядок URL вручну: так уникають помилок екранування й легше змінювати запит.

Коди відповіді. Статус-код каже, що сталося: `2xx` — успіх, `4xx` — помилка на боці клієнта (неправильний запит, немає ресурсу), `5xx` — збій на боці сервера. Реагувати на `4xx` і `5xx` варто по-різному: перше — привід виправити запит, друге — тимчасова проблема сервісу.

Надійність. Мережа ненадійна: сервер може бути недоступним або відповідати надто довго. Тому запит завжди роблять з обмеженням часу (**timeout**) і передбачають, що він може не вдатися. Застосунок від цього не має «падати».

JSON. Тіло відповіді зазвичай — JSON, який треба розібрати в структуру даних. Але покладатися на його форму наосліп не можна: потрібних полів може не бути, формат може відрізнятись від очікуваного — це теж треба обробити.

Відокремлення інтеграції. Код, що спілкується із зовнішнім API (знає про HTTP, коди, формат відповіді), варто тримати окремо від решти застосунку. Тоді веб-рівень працює з чистим результатом або зрозумілою помилкою й нічого не знає про деталі HTTP-клієнта. Цю саму межу ви пізніше проведете і для виклику AI API.

2. Завдання до виконання

Зробіть веб-застосунок, що за назвою міста показує поточну погоду (температуру й швидкість вітру). Джерело даних — зовнішній REST API **Open-Meteo** (без ключа доступу). Бекенд — на FastAPI, з простою сторінкою для вводу міста; роботу з API винесіть в окремий модуль.

Спосіб реалізації — на ваш розсуд. Ви можете писати код самостійно або скеровувати AI-асистента, але маєте **розуміти й уміти пояснити** кожне рішення.

Матеріали. В архіві до цієї роботи, папка `pr1`, вас чекає підготовлене середовище: `requirements.txt` із залежностями, скрипт `check_env.py` для перевірки налаштувань і доступу до Open-Meteo, каркас застосунку в папці `app` та збережені реальні відповіді обох сервісів у папці `samples`.

Каркас дає структуру проєкту й робочий веб-рівень, щоб заняття не пішло на плумбінг. Реалізації в ньому немає: функції модуля інтеграції — заготовки, обробки збоїв немає, і саме це ви й пишете. Зразки відповідей дозволяють відпрацювати розбір JSON, не звертаючись щоразу до мережі, і побачити наперед незручні випадки — зокрема те, що для ненайденого міста сервіс не повертає порожній список, а не повертає поля `results` взагалі.

Де вести роботу. Заведіть **власний репозиторій** на весь курс і розпакуйте архів у його корінь: усередині вже є папка `pr1/`, спільний `README.md` і `.gitignore`. Свій код пишіть у папці роботи, поруч із її матеріалами, і комітьте частинами — історія показує, як ви рухалися до рішення. Запропоновану структуру репозиторію описано в `README.md` з архіву.

Орієнтири.

- Open-Meteo: прогноз — `https://api.open-meteo.com/v1/forecast` (параметри: `latitude`, `longitude` та які поля погоди повертати); геокодування (назва міста → координати) — `https://geocoding-api.open-meteo.com/v1/search`.
- Доречні бібліотеки: `requests` (HTTP-клієнт), `FastAPI` + `uvicorn` (веб-рівень).

Продумайте самостійно.

- Як передати параметри запиту, не склеюючи URL вручну?
- Як застосунок має поводитися, коли сервер недоступний, відповідає надто довго або повертає помилку? Чи однаково реагувати на помилку клієнта (`4xx`) і сервера (`5xx`)?
- Що робити, якщо у відповіді немає очікуваних полів або тіло — не той JSON?
- Де провести межу між кодом інтеграції та рештою застосунку? Що веб-рівень має знати про `requests` ?
- Як користувач побачить помилку і який HTTP-статус поверне сервер?

3. Критерії приймання

- Запит виконується **з параметрами** й з обмеженням часу (**timeout**).
- Обробляються збої: недоступність, таймаут, коди `4xx` / `5xx`, несподіваний формат відповіді — застосунок **не аварійно завершується**.
- Робота з API винесена в **окремий модуль**; веб-рівень не містить деталей HTTP-клієнта й отримує від модуля готовий результат або зрозумілу помилку.
- Веб-ендпоінт повертає результат як **JSON** (успіх) або **відповідний HTTP-статус** (помилка); сторінка показує погоду чи зрозуміле повідомлення.

- Перевірено на кількох містах і на некоректному вводі (неіснуюче місто, порожній рядок).
- Ви можете **пояснити** кожне ухвалене рішення.

4. Контрольні запитання

1. Чому параметри краще передавати структуровано, а не склеювати в URL вручну?
2. Навіщо задавати timeout і як поводитися, коли він спрацював?
3. Як по-різному реагувати на 4xx і 5xx ?
4. Чому інтеграцію виносять в окремий модуль, а не тримають HTTP-виклики в коді інтерфейсу?
5. Які збої можливі під час звернення до зовнішнього API і як їх обробити?
6. Як цей самий підхід перенесеться на виклик AI API?

5. Рекомендована література

1. FastAPI Documentation. URL: <https://fastapi.tiangolo.com/> (дата звернення: 01.09.2026).
2. Requests: HTTP for Humans — Documentation. URL: <https://requests.readthedocs.io/> (дата звернення: 01.09.2026).
3. Open-Meteo. Free Weather API Documentation. URL: <https://open-meteo.com/en/docs> (дата звернення: 01.09.2026).
4. MDN Web Docs. HTTP response status codes. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status> (дата звернення: 01.09.2026).